11 Colecciones

Listas

Una **lista** en Python es una **colección ordenada y mutable de elementos** (Mutable significa que los elementos de una lista pueden ser modificados después de la creación de la lista).
Las listas son **una de las estructuras de datos más versátiles y más utilizadas**. Nos permiten almacenar colecciones ordenadas de elementos, como números, cadenas, u otros objetos.

Características de las Listas:

- **Mutabilidad**: Los elementos de una lista pueden ser modificados después de la creación de la lista
- **Ordenadas**: Los elementos de una lista están ordenados y mantienen el orden en el que se añadieron
- **Pueden contener cualquier tipo de dato**: Una lista puede contener elementos de diferentes tipos de datos, como enteros, cadenas, flotantes, u otras listas

Creación de una lista

Las listas se **definen utilizando corchetes []** y los elementos se separan por comas.

En este ejemplo, mi_lista es una lista que contiene los números del 1 al 5.

```
mi_lista = [1, 2, 3, 4, 5]
```

También es posible crear listas vacías, y posteriormente añadirle elementos según se necesite.

```
lista_vacia = []
```

11 Colecciones

Operaciones comunes con listas

Acceso a los elementos de una lista:

Los elementos de una lista se pueden acceder utilizando índices numéricos. El índice comienza en 0 para el primer elemento y se incrementa de forma consecutiva.

```
mi_lista = [10, 20, 30, 40, 50]

print(mi_lista[0])  # Resultado: 10
print(mi_lista[2])  # Resultado: 30
```

Agregar elementos:

Las listas en Python tienen varios métodos integrados para agregar elementos,

```
mi_lista.append(60)  # Agrega 60 al final de la lista
mi_lista.insert(2, 25)  # Inserta 25 en la posición 2
```

Concatenar listas

Se pueden realizar operaciones como la concatenación de listas o la multiplicación por un entero. Por ejemplo,

```
lista1 = [1, 2, 3]
lista2 = [4, 5, 6]

lista_concatenada = lista1 + lista2  # lista_concatenada es [1, 2,
3, 4, 5, 6]
lista_repetida = lista1 * 3  # lista_repetida es [1, 2, 3, 1, 2,
3, 1, 2, 3]
```

11 Colecciones

Eliminar elementos

Las listas en Python tienen varios métodos integrados para eliminar elementos,

```
mi_lista.remove(20)  # Elimina el valor 20 de la lista
mi_lista.pop(3)  # Elimina y devuelve el elemento en la posición 3
del mi_lista[1]  # Elimina el elemento en la posición 1
```

Ordenar

Las listas en Python tienen varios métodos integrados para ordenar elementos,

```
mi_lista.sort()  # Ordena la lista en orden ascendente
mi_lista_sorted = sorted(mi_lista)  # Crea una nueva lista
ordenada
```

Longitud

Las listas en Python tienen una función integrada para obtener su longitud,

```
longitud = len(mi_lista)  # Devuelve la longitud de la lista
```

Listas anidadas

En Python, es posible tener listas dentro de listas (esto se conoce como listas anidadas o listas multidimensionales).

Por ejemplo así,

```
lista_anidada = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
print(lista_anidada[0][1])  # Resultado: 2
```

11 Colecciones

Slices

Los slices en Python son una herramienta muy útil, que nos permite extraer porciones de secuencias de elementos (como listas, tuplas o cadenas) con muy poco código.

Los slices nos permiten trabajar con fragmentos específicos de una secuencia, sin tener que modificar ni iterar sobre la secuencia.

La sintaxis básica para crear un slice en Python es la siguiente:

```
secuencia[inicio:fin:paso]
```

- inicio: Índice donde comienza el slice (incluido)
- fin: Índice donde termina el slice (no incluido)
- paso: Tamaño del paso o incremento entre elementos del slice (opcional)

Consideraciones especiales

Omisión de valores valores

Podemos omitir uno, varios, o incluso todos los parámetros del Slice. Este se comportará distinto cada omisión.

Si no especificamos:

- El inicio, el slice comenzará desde el primer elemento
- El fin, el slice terminará en el último elemento
- El paso, el slice usará un paso de 1

```
primeros_salvo_tres = numeros[3:]  # Extrae los elementos del 3 al final
primeros_tres = numeros[:3]  # Extrae los tres primeros elementos
```

11 Colecciones

Uso de índices negativos

Python permite usar índices negativos para referirse a elementos relativos final de la secuencia (en lugar que desde el principio).

Parece un poco complicado, y al principio lía un poco. Pero básicamente,

```
ultimos_tres = numeros[-3:]  # Extrae los últimos tres elementos
primeros_salvo_tres = numeros[:-3]  # Extrae los elementos, salvo
los tres últimos
```

Por otro lado, si el paso, es negativo, el Slice recorre la secuencia en orden inverso.

Modificación de valores con slices

Los Slices también pueden ser utilizados para modificar valores en una secuencia.

```
frutas = ["manzana", "banana", "cereza", "dátil", "uva"]

frutas[1:3] = ["pera", "naranja"]  # Reemplaza "banana" y "cereza"
por "pera" y "naranja"
En este caso,

frutas[1:3] selecciona los elementos "banana" y "cereza"
Los reemplaza con "pera" y "naranja".
```

Ejemplos prácticos

Supongamos que tenemos una lista de números:

```
numeros = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

11 Colecciones

Ahora, veamos cómo el Slicing se comporta con diferentes combinaciones de inicio, fin y paso:

Sin especificar inicio, fin ni paso

```
resultado = numeros[:]
print(resultado)  # Resultado: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

En este caso, el slice copia toda la lista original, ya que no se especifica ningún valor para inicio, fin o paso.

Especificando inicio y fin

```
resultado = numeros[2:7]
print(resultado)  # Resultado: [2, 3, 4, 5, 6]
```

Aquí, el slice comienza en el índice 2 y termina en el índice 6 (no inclusivo), utilizando el paso predeterminado de 1.

Especificando solo fin

```
resultado = numeros[:5]
print(resultado)  # Resultado: [0, 1, 2, 3, 4]
```

En este caso, el Slice comienza desde el primer elemento y termina en el índice 5 (no inclusivo), utilizando el paso predeterminado de 1.

Especificando solo inicio

```
resultado = numeros[3:]
print(resultado)  # Resultado: [3, 4, 5, 6, 7, 8, 9]
```

Aquí, el Slice comienza en el índice 3 y continúa hasta el último elemento de la lista, utilizando el paso predeterminado de 1.

Especificando solo paso

```
numeros = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
numeros_pares = numeros[::2]  # Resultado: [0, 2, 4, 6, 8]
```

Aquí, numeros[::2] crea un nuevo slice de la lista numeros con un paso de 2, lo que significa que extrae cada segundo elemento, resultando en [0, 2, 4, 6, 8].

Especificando todo

```
resultado = numeros[1:8:2]
print(resultado)  # Resultado: [1, 3, 5, 7]
```

En este ejemplo, el slice comienza en el índice 1, termina en el índice 8 (no inclusivo) y utiliza un paso de 2, tomando cada segundo elemento desde el inicio hasta el final especificado.

Índices negativos al principio

Al utilizar índices negativos en el inicio del slice, se extraen elementos desde el final de la lista.

```
ultimos_tres = numeros[-3:]  # Extrae los últimos tres elementos:
[7, 8, 9]
```

En este caso, numeros[-3:] crea un slice que incluye los últimos tres elementos de la lista numeros.

Índices negativos al final

También es posible utilizar índices negativos al final del slice para definir el inicio desde el final de la lista.

```
sin_ultimos_tres = numeros[:-3]  # Extrae todos los elementos
excepto los últimos tres: [0, 1, 2, 3, 4, 5, 6]
```

Aquí, numeros[:-3] crea un slice que incluye todos los elementos de la lista numeros excepto los últimos tres.

El uso de un paso negativo en el slice invierte el orden de los elementos extraídos.

```
invertido = numeros[::-1]  # Invierte el orden de la lista: [9, 8,
7, 6, 5, 4, 3, 2, 1, 0]
```

Al especificar numeros[::-1], se crea un slice que comienza desde el último elemento hasta el primero, invirtiendo así el orden de la lista original.

Paso negativo

El uso de un paso negativo en el slice invierte el orden de los elementos extraídos.


```
invertido = numeros[::-1]  # Invierte el orden de la lista: [9, 8,
7, 6, 5, 4, 3, 2, 1, 0]
```


Al especificar numeros[::-1], se crea un slice que comienza desde el último elemento hasta el primero, invirtiendo así el orden de la lista original.

11 Colecciones

Métodos de colecciones

Python ofrece una serie de funciones y métodos integrados que facilitan la manipulación y el procesamiento de listas, tuplas y otros iterables

- map() Aplica una función a cada elemento de un iterable
- filter() Filtra elementos de un iterable basándose en una condición
- any() Retorna True si al menos un elemento del iterable es true
- all() Retorna True si todos los elementos del iterable son true
- sorted() Retorna una nueva lista ordenada a partir de los elementos de un iterable
- sum() Calcula la suma de los elementos de un iterable
- enumerate() Agrega un índice a cada elemento de un iterable
- reduce() Reduce una secuencia de elementos a un valor aplicando una función acumulativa
- zip() Combina elementos de múltiples iterables en tuplas

Funciones sobre iterables

Función map()

La función map() en Python se utiliza para aplicar una función a cada elemento de una secuencia y devolver un iterable con los resultados.

```
# Uso básico de map():
def cuadrado(x):
    return x ** 2

numeros = [1, 2, 3, 4, 5]
cuadrados = map(cuadrado, numeros)

print(list(cuadrados)) # [1, 4, 9, 16, 25]
```

En este ejemplo,

- map(cuadrado, numeros) aplica la función cuadrado() a cada elemento de la lista numeros,
- Devuelve un objeto map que contiene los cuadrados de los números.

11 Colecciones

En muchas ocasiones, emplearemos la función map() con una función lambda, para hacer el código más conciso:

```
# Uso de map() con función lambda:
numeros = [1, 2, 3, 4, 5]
cuadrados = map(lambda x: x ** 2, numeros)

print(list(cuadrados)) # [1, 4, 9, 16, 25]
```

Función filter()

La función filter() en Python se utiliza para filtrar elementos de una secuencia basándose en una condición definida por una función.

```
# Uso básico de filter():
def es_par(x):
    return x % 2 == 0

numeros = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
pares = filter(es_par, numeros)
```

En este ejemplo,

- filter(es_par, numeros) filtra los números de la lista numeros
- Devuelve un objeto filter que contiene solo los números pares

Al igual que con map(), también se puede utilizar una función lambda con filter():

```
# Uso básico de filter():
def es_par(x):
    return x % 2 == 0

numeros = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
pares = filter(es_par, numeros)

print(list(pares))  # Output: [2, 4, 6, 8, 10]
```

11 Colecciones

Funciones any() y all()

Las funciones any() y all() en Python son funciones booleanas que retornan True en función de los valores de un iterable

- any() si al menos un elemento es True
- all() si todos los elementos son True

(any()) o todos los elementos (all()) de un iterable son verdaderos, respectivamente.

```
# Uso de any() y all():
valores = [True, False, True, False, True]
hay_algun_verdadero = any(valores)  # True
todos_verdaderos = all(valores)     # False
```

Estas funciones son útiles para verificar condiciones en iterables (como listas o generadores).

En muchas ocasiones se emplean junto con map para aplicar una función a los valores de un iterable. Por ejemplo asi,

```
valores = [1, 2, 3, 4, 5]
algun_mayor_que_diez = any(map(lambda x: x > 10, valores))  #
False
```

Función sorted()

La función sorted() en Python retorna una nueva lista ordenada a partir de los elementos del iterable especificado.

```
# Uso básico de sorted():
numeros = [3, 1, 4, 1, 5, 9, 2, 6, 5, 3]
numeros_ordenados = sorted(numeros)

print(numeros_ordenados) # Output: [1, 1, 2, 3, 3, 4, 5, 5, 6, 9]
```

En este ejemplo, sorted(numeros) ordenará la lista numeros de forma ascendente y asignará el resultado a numeros_ordenados.

Opcionalmente, podemos especificar,

- Una función key para personalizar el ordenamiento
- Un argumento reverse para invertir el orden

```
# Uso de sorted() con key y reverse:
palabras = ["python", "java", "c++", "ruby", "javascript"]
palabras_ordenadas = sorted(palabras, key=len, reverse=True)

print(palabras_ordenadas) # ['javascript', 'python', 'java',
'c++', 'ruby']
```

Aquí, sorted(palabras, key=len, reverse=True) ordena la lista palabras por la longitud de las cadenas de forma descendente.

Función sum()

La función sum() en Python retorna la suma de los elementos del iterable especificado, opcionalmente comenzando desde un valor inicial especificado (por defecto, 0).

```
# Uso básico de sum():
numeros = [1, 2, 3, 4, 5]

suma_total = sum(numeros)  # 15
```

Este ejemplo suma todos los números en la lista numeros.

Función enumerate()

La función enumerate() en Python retorna un objeto enumerado que produce tuplas con un contador y un elemento del iterable especificado, opcionalmente comenzando desde un índice especificado (por defecto, 0).

```
# Uso básico de enumerate():
equipos = ["Real Madrid", "Barcelona", "Atlético de Madrid"]
enumeracion_equipos = list(enumerate(equipos, start=1))

print(enumeracion_equipos) # [(1, 'Real Madrid'), (2,
'Barcelona'), (3, 'Atlético de Madrid')]
```

En este ejemplo,

- enumerate(equipos, start=1) enumera los equipos de fútbol comenzando desde 1
- Genera una lista de tuplas (índice, equipo).

Función reduce()

La función reduce() en Python se utiliza para reducir una secuencia de elementos a un solo valor aplicando repetidamente una función de dos argumentos a los elementos de la secuencia.

```
# Uso básico de reduce():
from functools import reduce

def suma(x, y):
    return x + y

numeros = [1, 2, 3, 4, 5]
suma_total = reduce(suma, numeros)

print(suma_total) # Salida: 15
```

En este ejemplo,

- reduce(suma, numeros) aplica la función suma() de manera acumulativa a los elementos de la lista numeros
- Resulta en la suma total de todos los números.

11 Colecciones

Desde Python 3 la función reduce() se movió al módulo functools, por lo que es necesario importarla desde allí.

Función zip()

La función zip() en Python combina elementos de varios iterables en tuplas hasta que se agoten los elementos del iterable más corto.

```
# Uso básico de zip():
nombres = ["Luis", "María", "Pedro"]
edades = [30, 25, 35, 40]

print(personas) # [("Luis", 30), ("María", 25), ("Pedro", 35)]
```

En este ejemplo, zip(nombres, edades) combina las listas nombres y edades en una lista de tuplas (nombre, edad).

En este caso, devuelve una colección de 3 tuplas, porque nombres sólo tiene 3 valores. Cada tupla combina un valor del iterable nombres y del iterable edades.

11 Colecciones

Tuplas

Las tuplas son una colección de elementos ordenados e inmutables. Esto significa que una vez que se crea una tupla, sus elementos no pueden ser modificados, añadidos o eliminados.

Las tuplas son útiles cuando se necesita almacenar una colección de elementos que no van a cambiar a lo largo del tiempo.

Características de las Tuplas:

- Inmutabilidad: Una vez creada, no se pueden modificar sus elementos
- Ordenada: Los elementos de una tupla están ordenados y mantienen el orden en el que se añadieron
- Pueden contener cualquier tipo de dato: Una tupla puede contener elementos de diferentes tipos de datos

Además, las tuplas pueden ser utilizadas como claves en diccionarios, ya que al ser inmutables, garantizan que no cambiarán su valor a lo largo del tiempo.

Creación de una tupla

Las tuplas en Python se definen utilizando paréntesis ().

En este ejemplo, mi_tupla es una tupla que contiene los números del 1 al 5.

```
mi_tupla = (1, 2, 3, 4, 5)
```

También podemos usar la función tuple(), por ejemplo para convertir desde una colección.

```
mi_otra_tupla = tuple([1, 2, 3])
```

11 Colecciones

Operaciones comunes con tuplas

Acceso a elementos de una tupla

Para acceder a los elementos de una tupla podemos utilizar un índice numérico que empieza por 0 (de manera similar a como se accede a los elementos de una lista).

Por ejemplo, para acceder al primer elemento de la tupla mi_tupla, se utiliza el índice 0:

```
mi_tupla = (10, 20, 30, 40, 50)
print(mi_tupla[0])  # Resultado: 10
print(mi_tupla[2])  # Resultado: 30
```

Desempaquetado de tuplas

En Python, podemos asignar los elementos de una tupla a variables individuales en un solo paso (a esto se llama “desempaquetado de tuplas”).

```
mi_tupla = ("Luis", "Perez", 30)
nombre, apellido, edad = mi_tupla
print(nombre)  # Resultado: Luis
print(apellido)  # Resultado: Perez
print(edad)  # Resultado: 30
```

Ejemplos prácticos

Las tuplas son útiles en muchas situaciones. Veamos algunos ejemplos

Retorno múltiples valores

Cuando una función necesita devolver varios valores, puede utilizar una tupla para hacerlo de manera clara.

```
def obtener_datos_persona():
    nombre = "Luis"
    edad = 30
    return nombre, edad

datos = obtener_datos_persona()
nombre, edad = datos
```

Claves de Diccionarios

Las tuplas pueden ser utilizadas como claves en un diccionario porque son inmutables.

```
persona1 = ("Luis", 30)
persona2 = ("Ana", 25)

diccionario_personas = {persona1: "Programador", persona2:
"Diseñador"}
```

Iteración

Las tuplas se pueden utilizar en bucles for de manera similar a las listas.

```
mi_tupla = (10, 20, 30)
for numero in mi_tupla:
    print(numero)
```

11 Colecciones

Sets

Los Sets, o conjuntos, son estructuras de datos que nos permiten almacenar colecciones de elementos únicos y no ordenados.

Esto significa que no puede haber duplicados en un set y los elementos no están ordenados por posición

Características de los sets:

- Elementos Únicos: Los sets no pueden contener elementos duplicados, por lo que cada elemento es único
- Desordenados: Los elementos en un set no tienen un orden específico y no se garantiza que se mantenga el orden en el que fueron agregados

Creación de sets

Los conjuntos en Python se pueden crear utilizando llaves {}.
En este caso, se enumeran los elementos separados por comas dentro de las llaves para inicializar el conjunto.

```
mi_set = {1, 2, 3}
```

Crear un set con la función set()

También es posible crear conjuntos utilizando la función set(). Se pasa una lista como argumento a la función set(), y la función crea un conjunto con los elementos de esa lista.

```
mi_otro_set = set([1, 2, 3])
```

Por ejemplo, aquí hemos creado un Set {1, 2, 3} a partir de una Lista que contenía [1, 2, 3].

11 Colecciones

Operaciones con sets

Agregar elementos

Los conjuntos en Python tienen el método add() que se utiliza para agregar un solo elemento al conjunto.

```
mi_set.add(6)  # Agrega el elemento 6 al set
```

Eliminar elementos

Los conjuntos en Python tienen el método remove() que se utiliza para eliminar un elemento específico del conjunto.

Además, el método discard() también se puede utilizar para eliminar un elemento, pero no arrojará un error si el elemento no está presente en el conjunto.

```
mi_set.remove(3)  # Elimina el elemento 3 del set
mi_set.discard(2)  # Elimina el elemento 2 si está presente
```

Unión de sets

La unión de conjuntos se puede realizar utilizando el método union() o el operador |. Esto crea un nuevo conjunto que contiene todos los elementos de ambos conjuntos originales, eliminando duplicados.

```
union_set = mi_set.union(mi_otro_set)  # Une los dos sets en uno
nuevo
union_set = mi_set | mi_otro_set  # Lo mismo utilizando el
operador |
```

11 Colecciones

Intersección de sets

La intersección de conjuntos se puede realizar utilizando el método intersection() o el operador &. Esto crea un nuevo conjunto que contiene solo los elementos que están presentes en ambos conjuntos originales.

```
interseccion_set = mi_set.intersection(mi_otro_set)  # Obtiene la intersección de los sets
interseccion_set = mi_set & mi_otro_set  # Lo mismo utilizando el operador &
```

Diferencia de sets

La diferencia entre conjuntos se puede calcular utilizando el método difference() o el operador -. Esto crea un nuevo conjunto que contiene solo los elementos que están presentes en el primer conjunto pero no en el segundo.

```
diferencia_set = mi_set.difference(mi_otro_set)  # Obtiene la diferencia entre los sets
diferencia_set = mi_set - mi_otro_set  # Lo mismo utilizando el operador -
```

Ejemplos prácticos

Eliminación de duplicados en listas

Para eliminar duplicados de una lista en Python, se puede convertir la lista a un conjunto utilizando la función set().

```
lista_con_duplicados = [1, 2, 3, 4, 1, 2, 5]
conjunto_sin_duplicados = set(lista_con_duplicados)
```

11 Colecciones

Los Sets en Python no permiten elementos duplicados, por lo que al convertir la lista a un conjunto, automáticamente se eliminarán los duplicados.

Verificación de pertenencia

Para verificar si un elemento está presente en un conjunto, se puede utilizar la expresión in. Esta expresión devuelve True si el elemento está presente en el conjunto y False si no lo está.

```
if 3 in mi_set:
    print("El elemento 3 está presente en el set.")
```

Operaciones de conjuntos

Python ofrece varios métodos integrados para realizar operaciones de conjuntos, como unión, intersección y diferencia.

```
set1 = {1, 2, 3}
set2 = {3, 4, 5}

# Unión de sets
union = set1.union(set2)  # Resultado: {1, 2, 3, 4, 5}

# Intersección de sets
interseccion = set1.intersection(set2)  # Resultado: {3}

# Diferencia de sets
diferencia = set1.difference(set2)  # Resultado: {1, 2}
```

Estas operaciones se pueden realizar utilizando métodos de conjuntos como union(), intersection() y difference() (que hemos visto anteriormente).

11 Colecciones

Diccionarios

Un diccionario es una colección no ordenada de pares clave-valor. Las claves son únicas e inmutables, mientras que los valores pueden ser de cualquier tipo y pueden ser modificados.

Características de los diccionarios:

- Claves únicas: Cada clave en un diccionario es única, lo que significa que no puede haber claves duplicadas
- Mutabilidad de valores: Los valores de un diccionario pueden ser modificados después de la creación
- Desordenados: Aunque los diccionarios mantienen el orden de inserción a partir de Python 3.7, no se debe confiar en el orden

Creación de un diccionario

Los diccionarios en Python se definen utilizando llaves {} y los pares clave-valor se separan por dos puntos :. Por ejemplo, la sintaxis sería así,

```
mi_diccionario = {
    "clave1": valor1,
    "clave2": valor2,
    "clave3": valor3
}
```

Donde clave1, clave2 y clave3 son las claves que se utilizarán para acceder a los valores correspondientes.

Por ejemplo, asi.

```
mi_diccionario = {"nombre": "Luis", "altura": 30, "profesion":
"Ingeniero"}
```

En este ejemplo, mi_diccionario es un diccionario con tres pares clave-valor: - nombre con el valor “Luis”

- edad con el valor 30
- profesion con el valor “Ingeniero”

Operaciones con diccionarios

Acceso a elementos

Para acceder a los valores de un diccionario, se utiliza la clave correspondiente de la siguiente manera:

```
print(mi_diccionario["nombre"])  # Resultado: Luis
print(mi_diccionario["edad"])  # Resultado: 30
```

Modificación de elementos

Para modificar un diccionario, simplemente se asigna un nuevo valor a una clave existente o se agrega una nueva clave-valor de la siguiente manera:

```
mi_diccionario["clave1"] = nuevo_valor
mi_diccionario["nueva_clave"] = nuevo_valor
```

Agregar elementos

Para agregar un nuevo par clave-valor a un diccionario en Python, se puede utilizar la sintaxis mi_diccionario["nueva_clave"] = nuevo_valor.

```
mi_diccionario["ciudad"] = "Madrid"  # Agrega una nueva
clave-valor
```

Si la clave ya existe en el diccionario, el valor asociado se sobrescribirá con el nuevo valor proporcionado (como hemos visto en el apartado anterior).

Verificar pertenencia de claves

Para verificar si una clave está presente en un diccionario, se puede utilizar la expresión in. Esta expresión devuelve True si la clave está presente en el diccionario y False si no lo está.

```
if "nombre" in mi_diccionario:
    print("La clave 'nombre' está presente en el diccionario.")
```

Obtener claves y valores

El método items() retorna una vista de los pares clave-valor del diccionario en forma de tuplas.

```
mi_diccionario = {"nombre": "Luis", "edad": 30, "ciudad":
"Madrid"}
vista_items = mi_diccionario.items()  # Retorna una vista de los
pares clave-valor
print(vista_items)  # Resultado: dict_items([('nombre', 'Luis'),
('edad', 30), ('ciudad', 'Madrid')])
```

Por otro lado, también disponemos de los métodos:

- keys() devuelve una lista de todas las claves en el diccionario
- values() que devuelve una lista de todos los valores

```
claves = mi_diccionario.keys()  # Devuelve una lista de las claves
valores = mi_diccionario.values()  # Devuelve una lista de los
valores
```

Copia superficial del diccionario

El método copy() retorna una copia superficial del diccionario. Esto significa que se crea una nueva instancia de diccionario que contiene los mismos pares clave-valor que el diccionario original.

```
mi_diccionario = {"nombre": "Luis", "edad": 30, "ciudad":
"Madrid"}
copia_diccionario = mi_diccionario.copy()  # Crea una copia
superficial del diccionario

print(copia_diccionario)  # Resultado: {"nombre": "Luis", "edad":
30, "ciudad": "Madrid"}
```

Ojo que la copia es superficial!!. Si los valores del diccionario son referencias, los valores de la copia apuntarán a los mismos objetos.

Eliminar elementos

Hay varias formas de eliminar elementos de un diccionario en Python. Se puede utilizar la declaración del seguida del nombre de la clave para eliminar la clave y su valor correspondiente.

Alternativamente, el método pop() también puede ser utilizado para eliminar una clave específica y devolver su valor.

```
del mi_diccionario["profesion"]  # Elimina una clave y su valor
valor_eliminado = mi_diccionario.pop("edad")  # Elimina y devuelve
el valor de una clave
```

Eliminar todos los elementos del diccionario

El método clear() se utiliza para eliminar todos los elementos del diccionario, dejándolo vacío.

```
mi_diccionario = {"nombre": "Luis", "edad": 30, "ciudad":
"Madrid"}
mi_diccionario.clear()  # Elimina todos los elementos del
diccionario
print(mi_diccionario)  # Resultado: {}
```

11 Colecciones

Comprensión

Introducción

En Python, la comprensión (o comprehension en inglés) se refiere a una sintaxis concisa para crear nuevas secuencias a partir de iterables existentes.

Pueden aplicarse a cualquier iterable, lo que incluye listas, conjuntos y diccionarios, entre otros. Pero también funcionará su creamos nuestros propios iterables.

Las comprensiones no solo son más concisas en sintaxis si no que, en ocasiones, también pueden ser más rápidas que las construcciones equivalentes usando bucles for.

La sintaxis básica,

```
expresion for elemento in iterable
```

Opcionalmente, podemos pasar una condición de filtro así

```
expresion for elemento in iterable if condición
```

- expresion: Es la parte que define el resultado de la comprension. Es una operación que genera el elemento final a partir del original.
- for elemento in iterable: Este es el bucle que itera sobre cada elemento del iterable original
- if condicion: Esta es una condición opcional que filtra los elementos del iterable original, antes de aplicarles la expresion.

Esta sintaxis tiene variaciones mínimas en función del objeto que queramos general con la compresión (generalmente variará en si envolveremos entre [] o {})

Pero mejor, vamos a verlo con unos ejemplos:

11 Colecciones

Comprensiones de listas

Las comprensiones de listas permiten crear nuevas listas aplicando una expresión a cada elemento de una secuencia. La sintaxis básica es:

```
[expresion for elemento in iterable if condicion]
```

Por ejemplo, podemos aplicarlo sin condición

```
# Crear una lista de cuadrados de números del 0 al 9
cuadrados = [x**2 for x in range(10)]

print(cuadrados)  # Salida: [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

O en este ejemplo con una condición,

```
# Crear una lista de cuadrados de los numeros 0 al 9 que son pares
cuadrados_pares = [x**2 for x in range(10) if x % 2 == 0]

print(cuadrados_pares)  # Salida: [0, 4, 16, 36, 64]
```

Comprensiones anidadas

Las comprensiones de listas también pueden ser anidadas, lo que es útil para trabajar con estructuras multidimensionales.

```
# Crear una matriz identidad de 3x3
matriz_identidad = [[1 if i == j else 0 for j in range(3)] for i
in range(3)]

print(matriz_identidad)
# Salida: [[1, 0, 0], [0, 1, 0], [0, 0, 1]]
```

En este ejemplo,

- El [[...] for i in range(3)] exterior, genera 3 compresiones de lista
- El [1 if i == j else 0 for j in range(3)] interior genera una lista que tiene 3 elementos, que son 1 si i==j.

Comprensiones de conjuntos

Las comprensiones de conjuntos son similares a las de listas, pero generan un set. La sintaxis es:

```
{expresión for elemento in iterable if condición}
```

Por ejemplo,

```
# Crear un conjunto de números impares del 0 al 9
impares = {x for x in range(10) if x % 2 != 0}

print(impares)  # Salida: {1, 3, 5, 7, 9}
```

Comprensiones de diccionarios

También podemos usar compresiones que nos permiten construir diccionarios de manera concisa. La sintaxis es:

```
{clave: valor for elemento in iterable if condición}
```

Por ejemplo, sin condición,

```
# Crear un diccionario con números y sus cuadrados para números
del 0 al 9
cuadrados_dict = {x: x**2 for x in range(10)}

print(cuadrados_dict)
# Salida: {0: 0, 1: 1, 2: 4, 3: 9, 4: 16, 5: 25, 6: 36, 7: 49, 8:
64, 9: 81}
```

O con una condición,

```
# Crear un diccionario de cuadrados solo para números pares del 0 al 9
cuadrados_pares_dict = {x: x**2 for x in range(10) if x % 2 == 0}

print(cuadrados_pares_dict)
# Salida: {0: 0, 2: 4, 4: 16, 6: 36, 8: 64}
```

Comprensiones de generadores

Los generadores retornan un objeto que produce elementos. La sintaxis es:

```
(expresion for elemento in iterable if condicion)
```

Por ejemplo,

```
# Crear un generador de cuadrados de números del 0 al 9
cuadrados_gen = (x**2 for x in range(10))

print(list(cuadrados_gen))  # Salida: [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

Puede parecer similar al caso de una lista, pero en este caso lo que tenemos es un iterable de generadores.

Esto sería útil si trabajamos con grandes volúmenes de datos, que no necesitamos generar y cargar en memoria a la vez.

Es decir, en el ejemplo de la lista, cuadrados tenía calculados todos los valores. Sin embargo en el caso de cuadrados_gen se calcula una única vez cada vez que iteramos sobre la colección.

En el ejemplo parece similares, porque print(list()) recorre todos los elementos y los imprime.

11 Colecciones

Enumerate

La función enumerate() agrega un contador a una secuencia y devuelve una tupla que contiene el número de índice y el elemento correspondiente en cada iteración del bucle.

Es especialmente útil cuando necesitamos saber tanto el índice como el valor de cada elemento durante la iteración.

Sintaxis de enumerate()

La sintaxis general de enumerate() es:

```
enumerate(secuencia, inicio=0)
```

- secuencia: La secuencia sobre la cual iterar.
- inicio: Valor inicial del contador. Por defecto, es 0.

enumerate() es eficiente y optimizado para manejar grandes volúmenes de datos, ya que no crea una lista adicional en memoria sino que devuelve un iterador.

Es compatible con todo tipo de iterables en Python, incluyendo listas, tuplas, cadenas.

Ejemplo básico

En este ejemplo, enumerate se utiliza para recorrer una lista de nombres, imprimiendo el índice y el nombre de cada elemento.

```
nombres = ['Luis', 'María', 'Carlos', 'Ana']

for indice, nombre in enumerate(nombres):
    print(f"Índice {indice}: {nombre}")

# Salida:
```

```
# Índice 0: Luis
# Índice 1: María
# Índice 2: Carlos
# Índice 3: Ana
```

Ejemplos prácticos

Especificando un inicio para el contador

Aquí se muestra cómo usar enumerate especificando un valor de inicio diferente para el contador de índices. En este caso, el conteo comienza en 1 en lugar de 0.

```
nombres = ['Luis', 'María', 'Carlos', 'Ana']

for indice, nombre in enumerate(nombres, start=1):
    print(f"Estudiante {indice}: {nombre}")

# Salida:
# Estudiante 1: Luis
# Estudiante 2: María
# Estudiante 3: Carlos
# Estudiante 4: Ana
```

Utilizando enumerate() con otros tipos de datos

Este ejemplo muestra cómo enumerate también puede ser utilizado con otros tipos de secuencias, como cadenas de texto, para obtener el índice y el carácter en cada iteración.

```
cadena = "Python"

for indice, caracter in enumerate(cadena):
    print(f"Índice {indice}: {caracter}")

# Salida:
# Índice 0: P
```

```
# Índice 1: y
# Índice 2: t
# Índice 3: h
# Índice 4: o
# Índice 5: n
```

Creando un diccionario a partir de enumerate()

En este ejemplo, enumerate se usa en una comprensión de diccionario para crear un diccionario que mapea índices a valores de una lista de países.

```
paises = ['España', 'Francia', 'Italia']

diccionario_paises = {indice: pais for indice, pais in
enumerate(paises)}
print(diccionario_paises)

# Salida: {0: 'España', 1: 'Francia', 2: 'Italia'}
```

Frozen Sets

Un frozen set en Python es una colección desordenada e inmutable de elementos únicos. Se comporta de manera similar a un conjunto (set), pero no puede ser modificado una vez creado.

Es decir, que una vez creado no podemos agregar o eliminar elementos, ni modificar los existentes (intentar agregar, eliminar o modificar elementos generará un error).

Al igual que los set, los frozenset sólo pueden contener elementos únicos. Los elementos duplicados se eliminan automáticamente.

También igual que los set los elementos dentro de un frozenset no tienen un orden específico (por tanto, no se puede acceder a los elementos por índice, a diferencia de las listas).

Los frozen sets son útiles cuando **necesitamos una colección de elementos únicos que no se modificará** durante la ejecución del

programa. Por ejemplo, como claves en un diccionario o para incluirse en otros set (ya que ambos necesitan valores hashables e inmutables)

Creación de Frozen Sets

Los frozen set se crean utilizando la función frozenset() y pueden contener elementos de cualquier tipo hashable.

```
# Crear un frozenset a partir de una lista
conjunto_inmutable = frozenset([1, 2, 3, 4, 5])
print(conjunto_inmutable)  # Salida: frozenset({1, 2, 3, 4, 5})

# Crear un frozenset a partir de una lista de strings
frutas = frozenset(["manzana", "banana", "naranja"])

print(frutas)  # Salida: frozenset({'manzana', 'banana',
'naranja'})

# Crear un frozenset a partir de un set
conjunto_mutable = {1, 2, 3, 4, 5}
conjunto_inmutable = frozenset(conjunto_mutable)
print(conjunto_inmutable)  # Salida: frozenset({1, 2, 3, 4, 5})

# Crear un frozenset a partir de una cadena
conjunto_inmutable = frozenset('python')
print(conjunto_inmutable)  # Salida: frozenset({'p', 'y', 't',
'h', 'o', 'n'})
```

Operaciones con FrozenSet

A pesar de su inmutabilidad, los frozenset permiten realizar varias operaciones de conjuntos como unión, intersección y diferencia.

```
# Crear dos frozensets
set_a = frozenset([1, 2, 3])
set_b = frozenset([3, 4, 5])
```

```
# Unión de frozensets
union_set = set_a | set_b
print(union_set)  # Salida: frozenset({1, 2, 3, 4, 5})

# Intersección de frozensets
interseccion_set = set_a & set_b
print(interseccion_set)  # Salida: frozenset({3})

# Diferencia de frozensets
diferencia_set = set_a - set_b
print(diferencia_set)  # Salida: frozenset({1, 2})

# Diferencia simétrica de frozensets
diferencia_simetrica_set = set_a ^ set_b
print(diferencia_simetrica_set)  # Salida: frozenset({1, 2, 4, 5})

# Diferencia simétrica de frozensets
isSubet = frozenset([3, 4]) < set_b
print(isSubet)  # True
```

Aunque los frozenset permiten operaciones básicas de conjuntos, no soportan métodos que modifican el conjunto, como add(), remove(), o discard().

Ejemplos prácticos

Como claves en diccionarios

Dado que los frozenset son hashable e inmutables, pueden usarse como claves en diccionarios.

```
# Usar un frozenset como clave en un diccionario
diccionario = {frozenset([1, 2, 3]): "valor1", frozenset([4, 5,
6]): "valor2"}
print(diccionario[frozenset([1, 2, 3])])  # Salida: valor1
```

Elementos de otros Sets

Los frozenset pueden ser elementos de otros sets, lo que no es posible con los sets mutables.

```
# Usar frozensets como elementos de un set
set_of_frozensets = {frozenset([1, 2]), frozenset([3, 4])}

print(set_of_frozensets)  # Salida: {frozenset({1, 2}),
frozenset({3, 4})}
```

11 Colecciones

Ejercicios colecciones

Uniendo 2 listas en un diccionario

Creamos un diccionario a partir de 2 listas: una contendrá las claves y otra los valores:

```
keys = ['Ten', 'Twenty', 'Thirty'] #  Genera la lista keys
values = [10, 20, 30] # Genera la lista values

diccionario = {} #Genera un diccionario vacio.
contador = 0 # lo usaremos como índice para recorrer los valores
for key in keys: # itera la lista keys guardando en key un valor
cada vez
    diccionario[key] = values[contador] # añade al diccionario el
valor al que apunta "values[contador]" con clave "key"
    contador += 1 # incrementa el índice

print(diccionario)
```

Mezclar 2 diccionarios en uno

Unimos 2 diccionarios en uno que contendrá todos los pares:

```
dict1 = {'Ten': 10, 'Twenty': 20, 'Thirty': 30}
dict2 = {'Thirty': 30, 'Fourty': 40, 'Fifty': 50}

dict3 = dict1.copy()  # Creamos un nuevo diccionario
dict3.update(dict2)
print(dict1)
```

11 Colecciones

Acceso a un elemento dentro de un diccionario

Se pueden crear estructuras anidadas dentro de un diccionario: el valor es un nuevo diccionario.

Acceder al valor del elemento 'history' en un diccionario que tiene esta estructura.

```
sampleDict = {
   "class":{
      "student":{
         "name":"Mike",
         "marks":{
            "physics":70,
            "history":80
         }
      }
   }
}

sampleDict["class"]['student']['marks']['history']
```

Eliminar determinados pares de un diccionario

Vamos a ver cómo eliminar varios elementos de un diccionario:

```
diccionario = {
  "name": "Kelly",
  "age":25,
  "salary": 8000,
  "city": "New york"
}

clavesAQuitar = ["name", "salary"]
restantes = {}
```

11 Colecciones

```
for key in diccionario:
    if key not in clavesAQuitar:
        restantes[key] = diccionario[key]

print(restantes)
```